

3. ARITMÉTICA BINARIA.

(edición 2010)

Suma en binario.

Las reglas aritméticas para números binarios son sencillas. Las de suma son:

$$\begin{aligned} 0 + 0 &= 0 \\ 0 + 1 &= 1 \\ 1 + 0 &= 1 \\ 1 + 1 &= 10 \end{aligned}$$

A esto mismo lo podemos ver sobre una matriz.

+	0	1
0	0	1
1	1	10

Buscamos los operando en las coordenadas, y en la intersección de las mismas se encuentra el resultado.

Ej.: $1 + 1 = 10$

	+	0	↓ 1
	0	0	1
→	1	1	10

Al igual que en el sistema decimal cuando el resultado de la suma es de dos dígitos, se coloca el de menor peso (derecha) y el restante (acarreo) se suma a la columna siguiente.

Ej.:

$$\begin{array}{r} 11 \\ + 13 \\ \hline 24 \end{array} \qquad \begin{array}{r} \\ \\ \\ + \\ \hline \end{array} \begin{array}{l} \text{(línea de acarreo)} \\ 1011_2 \\ 1101_2 \\ \hline 11000_2 \text{ (resultado final).} \end{array}$$

Resta en binario.

Esta operación la podemos realizar por el método convencional, o sea operar en binario de la misma forma que lo hacemos en decimal.

Pero también la podemos resolver expresando la resta $a - b$, como una suma algebraica $a + (-b)$, convirtiendo de esta forma la resta en una suma. Para poder resolver la resta de esta forma, tendremos que definir antes los números negativos.

Representación binaria con signo.

Cuando un número binario es positivo, el signo se representa como 0 y la magnitud por el valor absoluto (que es el valor positivo). Cuando el número es negativo, el signo se representa por un 1 y el resto del número puede ser representado por cualquiera de las tres maneras siguientes:

1. Signo-magnitud
2. Signo-complemento de 1
3. Signo-complemento de 2.

En la representación signo-magnitud, esta última se representa con el valor absoluto (valor positivo). En las otras dos se representa con el complemento de 2 ó de 1.

Como ejemplo representamos el número binario 9 con signo, de las tres formas positivas y negativas sobre un registro de 8 bits.

	+	9	-	9
Signo-magnitud		0 0001001		1 0001001
Signo-complemento de 1		0 0001001		1 1110110
Signo-complemento de 2		0 0001001		1 1110111

Un número positivo en cualquier representación tiene un 0 en el bit de la extrema izquierda (codificando un +), seguido del número en valor absoluto (valor positivo). Un número negativo en la representación de signo-magnitud se obtiene complementando solamente el bit de signo. La representación complemento de 1 se obtiene complementando todos los bits a 1, (o sea reemplazar los 1 por 0 y 0 por 1) incluyendo el bit de signo. El complemento de 2 se obtiene sumándole 1 al complemento de 1 obtenido anteriormente.

Otra forma de hallar el complemento de 2 es restar el número al que queremos hallarle el complemento de la base elevada a la n , siendo n la cantidad de dígitos que tiene el número. Ej.: para 00001001_2 (8 dígitos), $n = 8$, $2^8 - 00001001_2 = 10000000_2 - 00001001_2 = 11110111_2$.

Si manejamos números con parte fraccionaria, lo mejor es tomar todo el número como entero (eliminamos la coma), operamos como anteriormente y luego proyectamos la coma al campo resultado. Ej.:

para 000010,01₂ (8 dígitos), n = 8, 2⁸ - 00001001₂ = 100000000₂ - 00001001₂ = 11110111₂, ahora en el campo resultado reponemos la coma y nos queda 111101,11₂. A partir del complemento de 2, se puede obtener el complemento de 1, restándole 1 al complemento de 2. En este caso también, si el número tiene parte fraccionaria, para hacer la resta debemos tomar toda la expresión como entera y luego de restarle 1, reubicar la coma. Ej.: Si el valor obtenido por complemento de 2 es 111101,11₂, tomamos 11110111₂, le restamos 1 y nos queda 11110110₂, ahora reubicamos la coma y obtenemos 111101,10₂.

Suma aritmética (suma o resta).

La representación de signo-magnitud es la que más se usa en los cálculos cotidianos. Por ejemplo, +23 y -35 son representados en un signo seguido por la magnitud del número. Para sumar estas dos funciones, es necesario restar la magnitud menor de la magnitud mayor y usar el signo del número mayor como signo del resultado, es decir (+23) + (-35) = -(35 - 23) es igual a -12. El proceso de sumar dos números con signo, cuando los números negativos están representados en la forma de signo-magnitud, requiere que se comparen estos signos. Si los dos signos no son iguales, se comparan las magnitudes relativas de los números y luego se resta el menor del mayor. Es necesario determinar también el signo del resultado. Esto una secuencia de controles que se hace compleja de resolver mediante un circuito digital.

Veamos ahora como se puede resolver estas operaciones sin necesidad de control alguno, cuando los números negativos están en función del complemento de 1 ó 2.

Suma representada por signo-complemento de 2.

La suma de dos números binarios con signo y los números negativos representados por sus complemento de 2 se obtienen de la suma de dos números con su bits de signo incluidos. Se descarta el arrastre en el bit más significativo. En el siguiente ejemplo se muestran las distintas posibilidades de suma, en donde los números negativos están expresados en complemento de 2 y por supuesto, si el resultado es negativo, también queda expresado en complemento de 2 y deberá ser complementado nuevamente a 2 para obtener el valor absoluto (valor positivo).

+6

0 000110₂

+ +9

0 001001₂

+15

0 001111₂

+6

0 000110₂

+ -9

1 110111₂

-3

0 1 111101₂

-6

1 111010₂

+ +9

0 001001₂

1 0 000011₂

-6

1 111010₂

+ -9

1 110111₂

-15

1 1 110001₂

Los dos números de los cuatro ejemplos se suman, con sus bits de signo incluidos. Cualquier arrastre de bit de signo se descarta y los resultados negativos se producen automáticamente en la forma de complemento de 2.

Suma representada por signo-complemento de 1.

La suma de dos números binarios negativos representados por sus complementos de 1, se obtienen de la suma de los números, con sus bits de signo incluidos. Si hay arrastre del bit más significativo (signo), el resultado se incrementa en 1 y el arrastre se descarta. El ejemplo anterior se repite, ahora resuelto con los números negativos representados por su complemento de 1. El arrastre del bit de signo se regresa y agrega al bit menos significativo.

+6

0 000110₂

+ +9

0 001001₂

+15

0 001111₂

+6

0 000110₂

+ -9

1 110110₂

-3

0 1 111100₂

-6

1 111001₂

+ +9

0 001001₂

1 0 000010₂

1

0 000011₂

-6

1 111001₂

+ -9

1 110110₂

1 1 101111₂

1

1 110000₂

Una de las ventajas de la representación en la forma signo-complemento de 2 sobre la forma signo-complemento de 1 y la forma signo-magnitud, es que la primera contiene un solo tipo de cero. Las otras dos representaciones tienen un cero negativo y otro positivo.

	+ 0	- 0
Signo-magnitud	0 0000000	1 0000000
Signo-complemento de 1	0 0000000	1 1111111
Signo-complemento de 2	0 0000000	ninguna

Esto ocurre debido a que el complemento de 2 de 0 0000000 (cero positivo) es 0 0000000 ya que el complemento a 1 es 1 1111111 y si le sumamos 1 y despreciamos el desborde 1 0 0000000, nos queda el mismo número.

Sobrecapacidad.

Si especificamos un número en complemento de 2, debemos tener en cuenta que el signo queda incluido dentro del mismo y que este se proyecta hasta el último dígito significativo del mismo.

Veámoslo con un ejemplo:

+3 en binario complemento de 2 es $0011_2 = 00011_2 = 0000011_2$
y -3 en binario complemento de 2 es $1101_2 = 11101_2 = 1111101_2$

Notamos que los ceros no significativos en los números positivos se convierten en unos (no significativos) en los números negativos.

Cuando dos números con n dígitos cada uno se suman y la suma ocupa n + 1 dígitos, se dice que hay un desbordamiento por sobrecapacidad.

El algoritmo para sumar dos números expresados en complemento de 2 como se ha establecido antes, produce un resultado incorrecto cuando se produce una sobrecapacidad. Esto es debido a que una sobrecapacidad de los bits del número cambian siempre el signo del resultado y se causa una respuesta errónea de n bits. La condición de sobrecapacidad se puede dar únicamente cuando ambos números son positivos o ambos negativos.

arrastre: 1

+35

0 100011₂

+40

0 010000₂

+75

1 001011₂

arrastre: 1 11

-35

1 011101₂

-40

1 011000₂

-75

0 110101₂

En ambos casos, se observa que el resultado, que debería ser positivo, es negativo o viceversa. Obviamente la respuesta es incorrecta y por lo tanto el algoritmo para sumar números binarios expresados en complemento de 2, falla cuando se produce una sobrecapacidad.

Una condición de sobrecapacidad puede ser detectada observando el arrastre a la posición del bit de signo y el arrastre de la posición del bit de signo. Si estos no son iguales se ha producido sobrecapacidad.

Para evitar los casos de sobrecapacidad debemos tener cuidado en trabajar con una cantidad de dígitos que sea capaz de albergar el campo resultado con signo incluido.

Los ordenadores digitales, detectan la sobrecapacidad y encienden una señal para indicar que el resultado no es correcto.

Multipliación binaria.

La multiplicación en binario es muy simple, debemos operar de la misma forma que en decimal, con la ventaja que solo tendremos que operar multiplicando por uno o cero.

1011₂ (11)

x 110₂ (6)

10110

1011--

100010₂ (66)

División Binaria.

Al igual que la multiplicación, para la división tenemos que proceder de la misma forma que conocemos para operar en decimal.

Veámoslo con un ejemplo: $10010_2 / 11_2$, paso por paso:

1) 1 0 0 1 0 | 11

- 1 1

1

2) 1 0 0 1 0 | 11

- 1 1

1 1

- 1 1

0

3) 1 0 0 1 0 | 11

- 1 1

1 1

- 1 1

0 0

0

Operaciones en otras bases.

La suma, resta, multiplicación y división en cualquier base dependen de la suma y la multiplicación. Por los tanto para facilitar el manejo de las mismas, veremos cómo construir una tabla de sumar y de multiplicar en cualquier base.

suma en base 5

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	10
2	2	3	4	10	11
3	3	4	10	11	12
4	4	10	11	12	13

multiplicación en base 5

x	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	11	13
3	0	3	11	14	22
4	0	4	13	22	31

Tabla 3-1. Suma y multiplicación en base 5

Se construye una matriz en cuyas coordenadas se colocan todos los números del sistema, y en la intersección de las mismas se coloca el resultado. Debemos tener en cuenta que cuando en una casilla hay un número de dos dígitos, se debe tomar el dígito de la derecha como resultado de la columna y el de la izquierda como acarreo.

Ejemplo de suma y multiplicación en base 5:

$$\begin{array}{r} 1 1 \text{ acarreos} \\ 3 1 4_5 \\ + 3 2 1_5 \\ \hline = 1 1 4 0_5 \end{array}$$

$$\begin{array}{r}
 \begin{array}{r}
 1 \\
 142_5 \\
 \times 32_5 \\
 \hline
 1 \\
 334
 \end{array} \\
 + \begin{array}{r}
 1 2 1 \\
 1031
 \end{array} \\
 \hline
 = 11144_5
 \end{array}$$

Se debe tomar como acarreo el dígito de mayor peso expresado en la tabla de multiplicar y debe ser sumado al valor de tabla en la operación siguiente.

En todos los sistemas numéricos, inclusive el decimal, la resta se puede realizar aplicando el complemento a la base del sistema para convertir un número en negativo y realizar luego la suma algebraica.

La división en otras bases se realiza de la misma forma que en decimal, esto es; se toma del dividendo el menor número de dígitos que sean mayor o igual al divisor, luego mediante multiplicaciones sucesivas, se busca la mayor cantidad de veces que el divisor entra en el dividendo, una vez logrado este valor, multiplica por el divisor y el resultado se resta del dividendo. Luego se anexan de a un dígito, operando de la misma forma hasta finalizar.

Ejemplo de división en base 5.

$$\begin{array}{r}
 1) \quad 4 \ 2 \ 1 \ 3_5 \\
 - 3 \ 2 \\
 \hline
 1 \ 0
 \end{array}$$

$$\begin{array}{r}
 2) \quad 4 \ 2 \ 1 \ 3_5 \\
 - 3 \ 2 \downarrow \\
 \hline
 1 \ 0 \ 1 \\
 3 \ 2 \\
 \hline
 1 \ 4
 \end{array}$$

$$\begin{array}{r}
 3) \quad 4 \ 2 \ 1 \ 3_5 \\
 - 3 \ 2 \\
 \hline
 1 \ 0 \ 1 \\
 - 3 \ 2 \downarrow \\
 \hline
 1 \ 4 \ 3 \\
 - 1 \ 1 \ 4 \\
 \hline
 2 \ 4
 \end{array}$$

Construir las tablas de suma y multiplicación para los sistemas octal y hexadecimal.

Operaciones aritméticas en BCDN.

Considere la suma aritmética de dos dígitos decimales en BCDN con un arrastre posible de un estado anterior. Como cada dígito no puede ser mayor que nueve, la salida no puede superar $9 + 9 + 1 = 19$, siendo el 1 en la suma, un posible arrastre anterior. Si sumamos dos dígitos, el resultado entonces puede variar entre 0 y 19. Si se analiza el resultado veremos que si este es menor o igual a 1001, (9) es correcto en BCDN, pero, si es mayor, no coincide con el valor real. Esto se debe a que en BCDN los números binarios de 1010 al 1111 no existen, ya que de la combinación correspondiente al (9) 0000 1001 pasamos al (10) 0001 0000. Por lo tanto luego de operar con un dígito BCDN, si el resultado es mayor que 1001 (9), tendremos que realizar un reajuste de 0110 (6) unidades para lograr el resultado correcto. En este caso siempre tendremos acarreo.

Ejemplo:

a)

$$\begin{array}{r} 40 \quad 0100 \ 0000 \\ + 21 \quad 0010 \ 0001 \\ \hline 61 \quad 0110 \ 0001 \end{array}$$

b)

$$\begin{array}{r} 174 \quad 0001^1 \quad 0111^1 \quad 0100 \\ + 398 \quad 0011 \quad 1001 \quad 1000 \\ \hline 0101^1 \quad 0001 \quad 1100 \\ \quad \quad 0110 \quad 0110 \\ \hline 572 \quad 0101 \quad 0111^1 \quad 0010 \end{array}$$

Resta en BCDN.

La resta en el sistema decimal se puede realizar de una forma similar al sistema de base dos, sumando al minuendo el complemento a diez (base de sistema) o a nueve (complemento a la base - 1) del sustraendo. En los sistemas BCD se utiliza normalmente el complemento a nueve porque permite reducir la complejidad del sistema.

Realizando la operación de esta forma debemos tener en cuenta que si el resultado es negativo, no se produce acarreo y el número obtenido queda representado mediante su complemento a nueve. Si por el contrario el resultado es positivo, se produce un acarreo que ha de sumarse al resultado para obtener el resultado correcto.

Como ejemplo realizaremos la resta $24 - 17$:

$$\begin{array}{r} \text{en decimal: } 24 \\ - 17 \\ \hline = 07 \end{array}$$

en decimal por
complemento de 9

$$\begin{array}{r} 24 \\ + 82 \\ \hline 106 \\ \text{L} \rightarrow 1 \\ \hline = 07 \end{array}$$

en código BCDN tendremos:

$$\begin{array}{r} 1 \quad 0010 \quad 0100 \\ - \quad 1000 \quad 0010 \\ \hline 1010 \quad 0110 \\ 0110 \end{array} \quad \begin{array}{l} (24) \\ (82) \text{ comp. a 9 de 17} \end{array}$$

$$\begin{array}{r} 1 \overline{) 0000 \ 0110} \\ + \quad \quad \quad \rightarrow 1 \\ \hline 0000 \ 0111 \end{array} \quad (07)$$

Ahora realicemos 17 - 24 = -7

en decimal:

$$\begin{array}{r} 17 \\ - 24 \\ \hline = -07 \end{array}$$

en decimal por
complemento a 9:

$$\begin{array}{r} 17 \\ + 75 \\ \hline = 92 \end{array}$$

92 es el complemento a nueve de 07.

en BCDN tendremos:

$$\begin{array}{r} 1111 \overline{) 0001} \quad \begin{array}{l} (17) \\ (75) \end{array} \\ + 0111 \quad \begin{array}{l} 0111 \\ 0101 \end{array} \\ \hline 1001 \quad \begin{array}{l} 1100 \\ 0110 \end{array} \\ \hline = 1001 \ 10010 \end{array} \quad \begin{array}{l} \text{corrección} \\ (92) \end{array}$$

Suma en BCD ex 3.

Al igual que en BCDN, en BCD ex 3, también se deberá ajustar cada resultado en seis cada vez que este supere el valor 9 y teniendo en cuenta que 9 en BCD ex 3 es 1100. Además, si observamos la suma de dos números excedidos en 3, veremos que el resultado queda excedido en seis por lo tanto también tendremos siempre que hacer un reajuste en -3 para lograr el resultado correcto.

Ahora bien, el reajuste de +6 se debe hacer en los resultados mayores que 1100 (9), si este resultado se encuentra excedido en 3 llegaría a 1111, por lo tanto un valor mayor genera un desborde. Si nos valemos de este indicador podemos decir que si luego de la suma, se generó un desborde el resultado debe ser reajustado en +6 (por ser mayor que 9) y en -3 (por estar doblemente excedido en 3), o sea que solo sumamos 3. Si no se generó desborde, el resultado no debe ser reajustado en +6, pero si en -3, en este caso se resta 3.

Ejemplo:

en decimal	$\begin{array}{r} 46 \\ + 18 \\ \hline \end{array}$	en BCD ex 3	$\begin{array}{r} 1 \overline{) 0111} \quad \begin{array}{l} 1001 \\ 1011 \end{array} \\ + 0100 \quad \begin{array}{l} 1100 \ 10100 \\ - 11 \quad + 11 \end{array} \\ \hline = 1001 \ 0111 \end{array}$
	= 64		

Resta en BCD ex 3.

La resta en BCD ex 3 se realiza de la misma forma que en BCDN, por complemento de 9 y recordemos que el código BCD ex 3 tiene la característica de ser autocomplementario, por lo tanto será fácil lograr el complemento, cambiando los unos por ceros y los ceros por unos.

Cuando se estudien los circuitos sumadores, se notara una reducción en el circuito que opera en BCD ex 3 respecto del similar para BCDN.